General Functions

data()

This function loads a data set, which is often a built-in part of R.

```
> data(iris)
>
```

This command loads the data.frame "iris" into your RAM and then lets you type your next command into the terminal. There will be no further feedback from the shell as is often usual with other commands.

library()

This function makes an add-on package available.

```
> library(stringr)
> library(dplyr)
> library(tidyverse) # This library contains ggplot2
>
```

This command loads the data.frame "iris" into your RAM and then lets you type your next command into the terminal. There will be no further feedback from the shell as is often usual with other commands.

Data Import

read_csv()

```
# You can either browse to the desired folder as shown below, or setwd()
# to the relevant directory and directly type in the file name, without apostrophes.

> read_csv("data/file1.csv")
>
```

Data Cleaning

str_split()

```
> library(stringr)
> str_split("banana", pattern= "a")
[[1]]
[1] "b" "n" "n" ""
```

filter(data.frame(#column_name = c(), #repeat_n_times), range)

```
# The last row rounds values in the numbers column to the second decimal digit and
    replaces the decimal mark

> library(dplyr)
>

RScript:
df <- data.frame(id = c(1:3,1:3),
                 numbers= c(1.456, 5.784356, 3.123, 8.234, 23.5465, 2.345),
                 year=c(2017,2017,2017,2018,2018,2018))
df_filtered <- filter(df, numbers < 5 & year== 2017)
numbers_formatted <- format(round(df_filtered$numbers, 2), decimal.mark=",")
```

summarise(filter(data.frame(#column_name = c(), #repeat_n_times), range)

```
df_c <- data.frame(id = c(1:3,1:3),
                   money_spent= c(1000, 2000, 6000, 1500, 3000, 5500),
                   currency = c("CHF", "CHF", "USD", "EUR", "CHF", "USD"),
                   year=c(2017,2017,2017,2018,2018,2018))
df_filtered <- filter(df_c, currency== "CHF" )
df_summary <- summarise(df_filtered,
                        total = sum(money_spent),
                        N_obs = n())
```

mutate()

The mutate() function in R is used to create new variables or modify existing variables in a data frame without removing any other variables. It allows you to apply transformations or calculations to columns and add the result as new columns or overwrite existing ones.

```
library(dplyr)

d <- data.frame( name = c("Abhi", "Bhavesh", "Chaman", "Dimri"),
                 age = c(7, 5, 9, 16),
                 ht = c(46, NA, NA, 69),
                 school = c("yes", "yes", "no", "no") )

mutate_d <- mutate(d, x3 = ht + age)
```

Data Visualisation

ggplot()

Every ggplot2 plot has at least three key components: data, aes, and a geom function (or another rendering-instruction):

```
> data(iris)
> library(ggplot2)
> ggplot(iris, aes(x=Sepal.Length, y=Sepal.Width)) + geom_point()
>
```

Typical Errors

Error 1 (Lack of attributes): You can see that the third attribute is missing, which is why there will be a plot, however it is going to be empty:

```
> data("swiss")
> library(tidyverse)
> ggplot(swiss, aes(x=Agriculture, y=Examination))
```

Error 2 (Faulty configuration of functions): This is also not going to work for several reasons: (1) The data.frame has not been loaded into R, (2) since there is no df-object, the axes "A" and "B" will also have no meaning to the aes-attribute within the ggplot-function, and (3) there is no color "C" in the geom_point function's aesthetics.

```
> ggplot(df, aes(x=A, y=B)) + geom_point(aes(colour=C)) + geom_smooth(colour="red", method
    = "lm")
>
```

Assume that there is an object called "df", then:

1. The line geom_smooth(colour="red", method = "lm") adds an additional layer to the plot, displayed on top of what geom_point(aes(colour=C)) generates.
2. df must be a data.frame, tibble, or data.table with columns A, B, and C in order to make this code work (result in a plot).
3. The resulting plot will have a legend.